Luca Danisi **20-3-2026**

Exploit DVWA – XSS e SQL injection

Scegliere una delle vulnerabilità **XSS** ed una delle vulnerabilità **SQL** injection: lo scopo del laboratorio è sfruttarle con successo.

🧪 **IL NOSTRO LABORATORIO**

🐉 **KALI** : rete interna vtnet 1 → IP 192.168.50.100

💀 **Metasploitable** : rete interna vtnet1 → IP 192.168.50.101

Step 1.1 – Comunicazione Macchine Virtuali

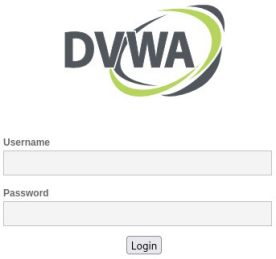
Verifichiamo che le nostre macchine siano in grado di comunicare correttamente effetuando un ping.

```
┌──(kali㉿kali)-[~]
└─$ ping 192.168.50.101
PING 192.168.50.101 (192.168.50.101) 56(84) bytes of data.
64 bytes from 192.168.50.101: icmp_seq=1 ttl=64 time=6.87 ms
64 bytes from 192.168.50.101: icmp_seq=2 ttl=64 time=2.74 ms
64 bytes from 192.168.50.101: icmp_seq=3 ttl=64 time=2.59 ms
^C
--- 192.168.50.101 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2093ms
rtt min/avg/max/mdev = 2.589/4.066/6.873/1.985 ms
```

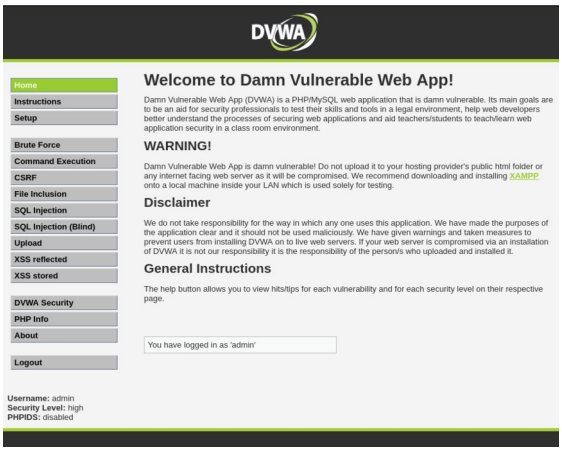
"Comunicazione avvenuta con successo"

Step 1.2 – Accesso e configurazione DWA

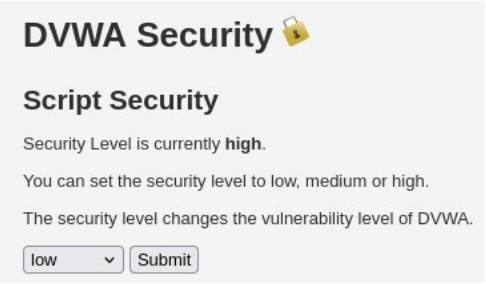
Utilizzando il browser su Kali apriamo la **DVWA** inserendo l'indirizzo **http://192.168.50.101**.



Entriamo con le credenziali di default : "admin-password"

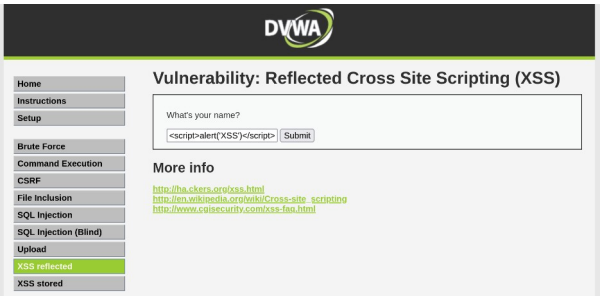


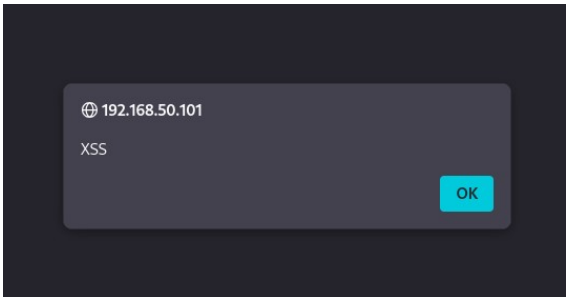
Impostiamo il security level su **LOW**



Step 2.1 – **XSS Reflected**

Nella DVWA scegliamo il menu XSS Reflected e inseriamo lo script **"<?<script>alert('XSS')</script>"** , Il browser lo interpreta come codice JavaScript ed esegue un popup di alert

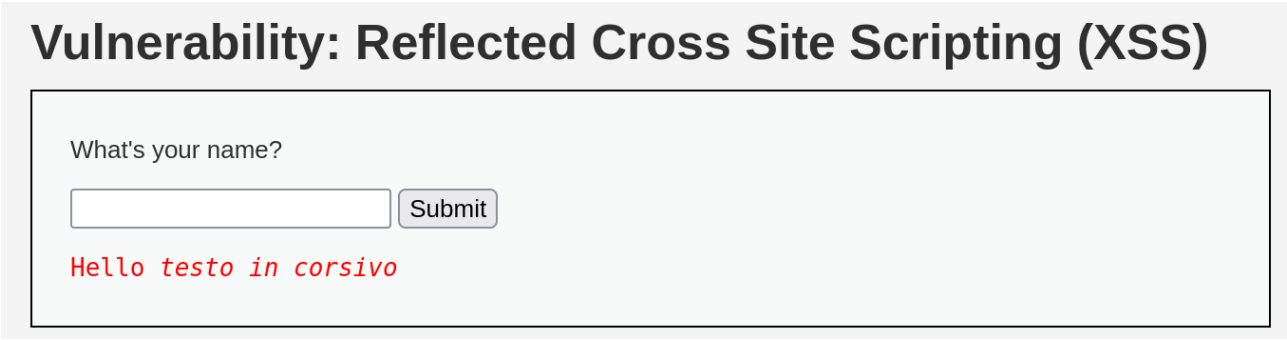




XSS confermato! Il popup mostra "XSS" proveniente da `192.168.50.101` — vulnerabilità verificata.

Step 2.2 – Test con Tag HTML - CORSIVO

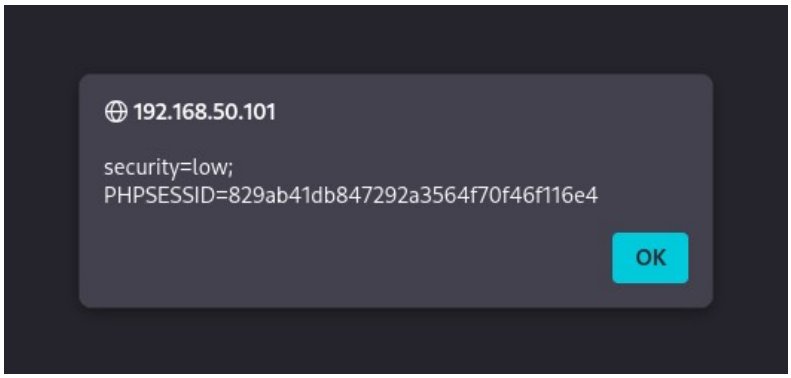
Questa volta inseriamo lo script **“testo in corsivo”** , il browser mostrerà il testo in corsivo nella pagina, confermando che l exploit della vulnerabilità.



Step 2.3 – Recupero cookie di sesisione

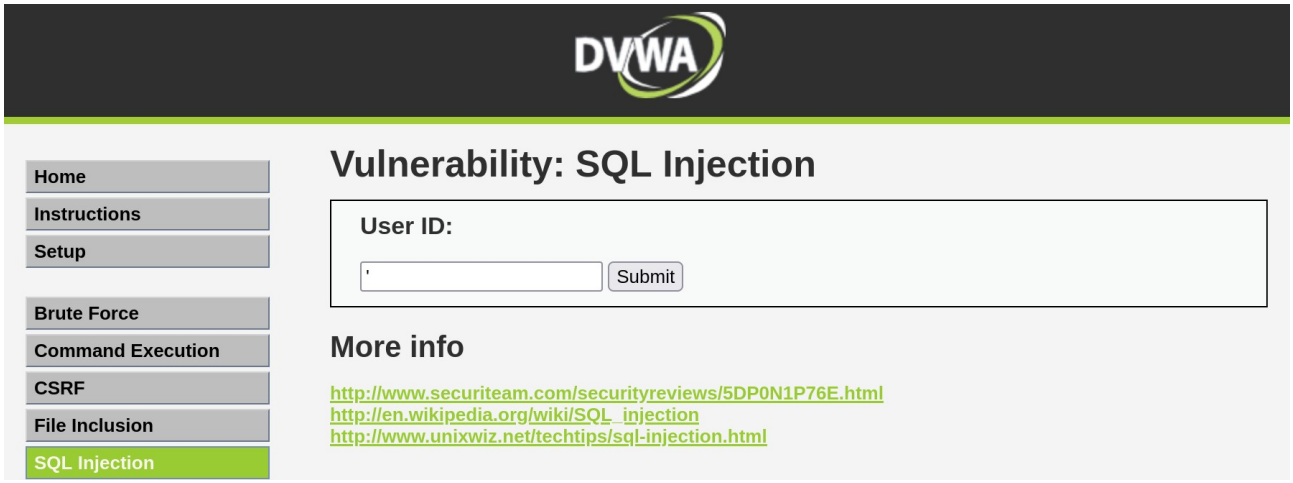
A Inserendo lo script **“<script>alert(document.cookie)</script>”**, forziamo il browser a mostrarci il cookie di sessione dell’utente.

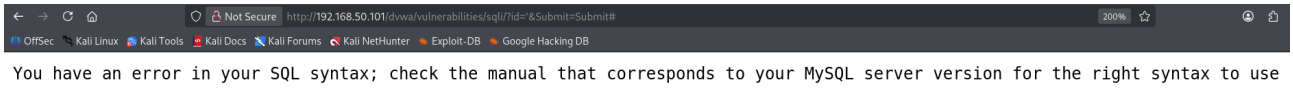
In uno scenario reale, un attaccante potrebbe rubare questo cookie per impersonare la vittima



3.1 – **SQL Injection**

Nel campo User ID inseriamo una query “ ‘ “ , per testare se il campo è vulnerabile, il browser ci restituirà un errore SQL , confermando che l input non viene filtrato.

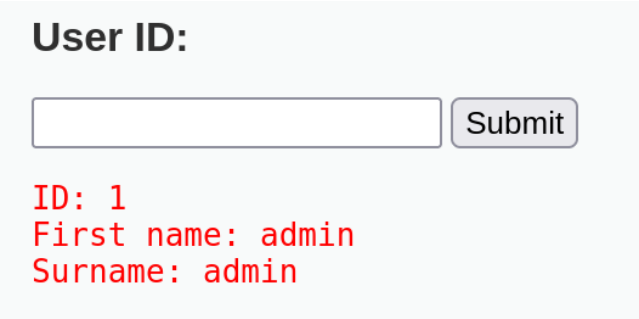




Il database sta espondendo i suoi errori , mostrandoci l utilizzo di MySQL

3.2 – Query leggittima

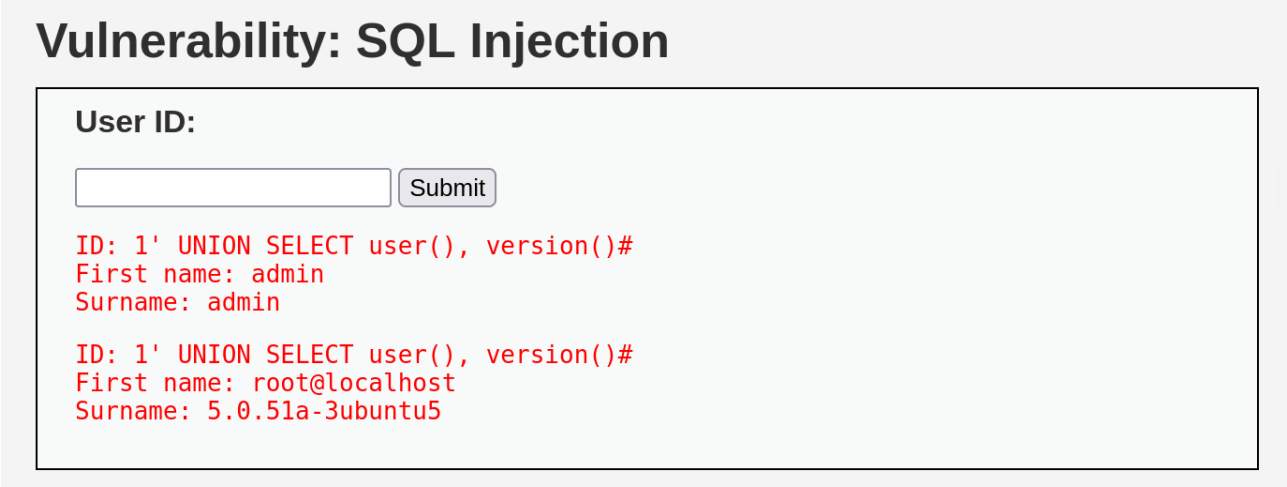
Inseriamo un ID valido “ **1** “ per vedere come risponde normalmente alla query, così capiamo la struttura dell'output prima di manipolarlo.



Sappiamo che il database ha 2 colonne (first name e Surname)

3.2 – Union attack

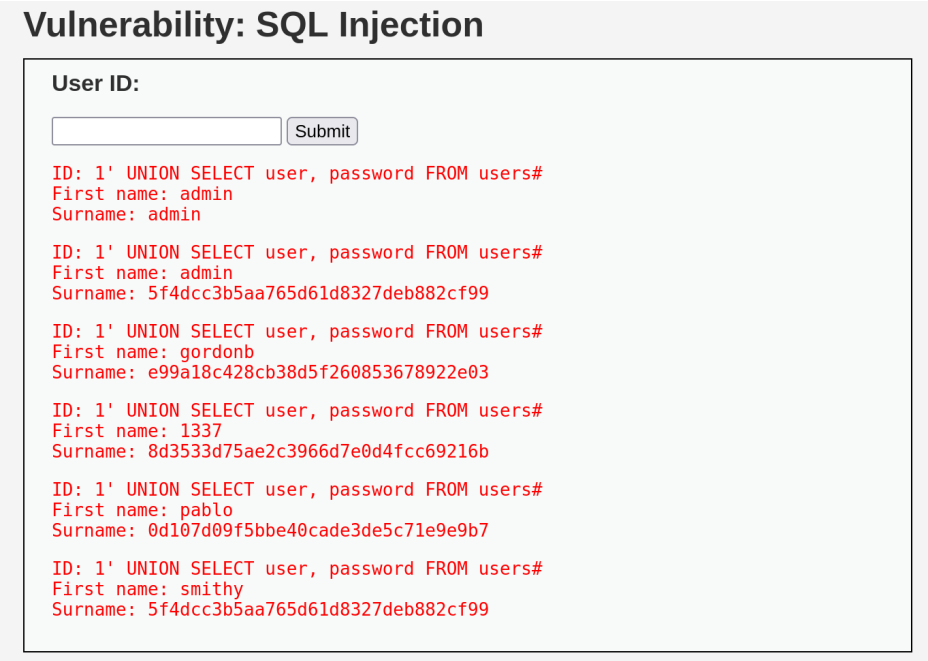
Con il seguente comando **“1' UNION SELECT user(), version()#”**, stiamo interrogano l user e la versione.



Il database gira come root,e sappiamo anche la versione utilizzata

4.2 – Estrarre tutto il database

Con la query **“1' UNION SELECT user, password FROM users#”**, interroghiamo direttamente il software db , chiedendo di fornirci l intero database.



5 – CONCLUSIONI

Il laboratorio ha dimostrato come due delle vulnerabilità più comuni e pericolose del web — XSS Reflected e SQL Injection — possano essere sfruttate con successo su un'applicazione priva di **FIltering** dell'input.

Tramite **XSS Reflected** è stato possibile iniettare codice JavaScript arbitrario nella pagina, arrivando a recuperare il cookie di sessione dell'utente (`PHPSESSID`), che in uno scenario reale permetterebbe a un attaccante di effettuare **session hijacking** (ovvero impersonare la vittima senza conoscerne la password).

Tramite **SQL Injection non blind** è stato possibile, partendo da un semplice apostrofo, risalire alla struttura del database, alla versione di MySQL, all'utente con cui gira il servizio (`root@localhost`) e infine estrarre l'intera tabella utenti con relative password .

La causa comune di entrambe le vulnerabilità è la stessa: l'applicazione si fida dell'input dell'utente senza filtralo.